Feed-forward Neural Networks for Regression and Classification

FYS-STK3155 Project 2

Eirik Bell-Mathiesen
University of Oslo
eiribel@uio.no

4 November 2026

Abstract

We compared a custom-implemented feed-forward neural network with polynomial models for fitting noisy observations of the Runge function. We also compared the network with logistic regression for classifying AL-PUB v2 letter images into 24 classes. Adam gave the lowest Runge validation mean squared error (MSE) among the tested optimiser setups. On the test set, the validation-selected neural network achieved an MSE of 0.011367, compared with 0.011217 for the degree-14 OLS model. Increasing network size sometimes widened the gap between training and validation errors. The selected L1- and L2-regularised networks had validation MSE similar to the selected unregularised network. For letters, the custom network achieved test macro-F1 0.5086 and accuracy 0.6092, compared with 0.3873 and 0.4986 for logistic regression. The library-implemented network achieved similar classification scores. The neural network outperformed logistic regression on letter classification, while the degree-14 polynomial model achieved comparable test MSE to the neural network on Runge regression. The comparisons concern the tested architectures and hyperparameters, with the stated number of epochs.

1 Introduction

Neural networks can learn features from data, but this flexibility does not guarantee better predictions than simpler models. We examine whether learned features improve predictive performance relative to simpler models in two contrasting tasks. The first task fits noisy observations of a known smooth function, the Runge function. Polynomial features provide a natural fixed-feature baseline for this regression task. For handwritten Greek letters from ancient papyri, the network learns patterns from pixel values. Within each task, we also assess how training choices affect generalisation, or predictive performance on unseen data.

Building on Project 1's polynomial models, we implemented a shared feed-forward neural network (FFNN) for regression and classification. It learns intermediate features through hidden-layer weights and biases, using the course's forward pass to compute predictions and backward pass to compute the gradients used in training (Hjorth-Jensen 2026, Ch. 8).

For noisy Runge regression, polynomial ordinary least squares (OLS) provides the fixed-feature baseline, with Ridge and Lasso providing regularised alternatives that penalise coefficient magnitude. For classification, we used AL-PUB v2, a dataset of images of handwritten Greek letters from ancient papyri. Recognising these letters can aid manuscript recovery (Swindall et al. 2021). Logistic regression provides a linear classification baseline for comparing predictions with those of a network that can learn nonlinear features. A scikit-learn FFNN provides an additional implementation benchmark for our custom network. Comparing parameter-update algorithms (optimisers) and network size (capacity) helps us assess how training choices and model complexity affect predictions.

Section 2 introduces the theoretical background. Section 3 describes the methods used in the experiments. Section 4 presents the results, and Section 5 discusses the findings. The working notebook is `Code/Project2_working.ipynb`. Appendix A gives the search ranges and links to the supplementary instructions for reproducing the results.

2 Theory

2.1 From fixed features to learned features

Supervised learning fits predictions from observations with input features x_i and known targets y_i, then assesses generalisation on unseen observations (Goodfellow et al. 2016, Ch. 5, Sec. 5.2). This section gives the theoretical basis for comparing fixed-feature and learned-feature models, including their training and predictive assessment.

The regression models differ in how they construct their input features. Polynomial regression fixes those features as powers of x:

$$\hat{y}(x) = \theta_0 + \sum_{j=1}^{p} \theta_j x^j.$$

The degree p sets the highest power, and the fitted coefficients θ_j weight the fixed features. The model is linear in those coefficients even though its prediction is nonlinear in x. A design matrix stores observations in rows and features in columns (Hjorth-Jensen 2026, Ch. 3, design matrix and centring).

A feed-forward neural network (FFNN) learns features through hidden layers. With a batch $A^0 = X$, its layer calculations are

$$Z^l = A^{l-1}W^l + b^l, \qquad A^l = f_l(Z^l), \qquad l = 1, \ldots, L. \tag{1}$$

Here X has m rows and $d = n_0$ input columns. Layer l has n_l outputs, and L includes the output layer. W^l has shape $n_{l-1} \times n_l$. Bias b^l has n_l entries and is added to each row, and f_l maps preactivations Z^l to outputs A^l. Hidden-layer width is the number of neurons, and depth is the number of hidden layers. One hidden layer with a scalar linear output gives

$$\hat{y}(x) = b^{\text{out}} + \sum_{j=1}^{H} v_j f\left(b_j + \sum_{r=1}^{d} x_r w_{rj}\right).$$

Here H is the hidden width and v_j weights hidden feature j. These intermediate features change during training as their incoming weights and biases are fitted along with the output weights.

With enough width, one sigmoid hidden layer and a linear output can approximate any continuous scalar function on a compact domain to arbitrary uniform accuracy (Hjorth-Jensen 2026, Secs. 8.4–8.5).

2.2 Hidden activation functions

Hidden activation functions let the network transform its inputs nonlinearly, and their derivatives are needed to propagate gradients. Nonlinear activation functions are necessary for the network to represent more than an affine transformation, since composing affine layers without them produces another affine transformation (Goodfellow et al. 2016, Ch. 6, introduction). The hidden activations and their derivatives are

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)[1-\sigma(z)], \tag{2}$$

$$r(z) = \max(0, z), \qquad r'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0, \end{cases} \tag{3}$$

$$\ell(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases} \qquad \ell'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z < 0. \end{cases} \tag{4}$$

Sigmoid saturates at large $|z|$, where its small derivative reduces the gradient passed to earlier layers. ReLU has a slope of one for positive inputs and zero for negative inputs, meaning that negative inputs prevent gradients from passing backward through the neuron during training. Leaky ReLU keeps a negative-branch slope, with $0 < \alpha < 1$. Both ReLU functions are nondifferentiable at zero, so evaluating their gradients there requires an assigned convention (Hjorth-Jensen 2026, Sec. 8.6).

2.3 Output activations and loss functions

The activation function in the output layer determines the possible prediction values and helps define how those values are interpreted. The loss function measures disagreement between predictions and targets, so the output and loss must suit the task and work together. Matched pairs are an identity (linear) output with MSE for regression, sigmoid with binary cross-entropy for binary classification, and softmax with categorical cross-entropy for mutually exclusive classes. For a linear regression output $\hat{Y}$, define full mean squared error (MSE) over m observations and K output entries per observation, with $K = 1$ for scalar regression:

$$C_{\text{MSE}} = \frac{1}{mK}\sum_{i=1}^{m}\sum_{k=1}^{K}(\hat{y}_{ik} - y_{ik})^2, \qquad \frac{\partial C_{\text{MSE}}}{\partial \hat{y}_{ik}} = \frac{2}{mK}(\hat{y}_{ik} - y_{ik}). \tag{5}$$

With independent fixed-variance Gaussian noise, minimising squared error maximises the likelihood of the observed targets (Hjorth-Jensen 2026, Ch. 3, Gaussian likelihood). The full-MSE convention accounts for the factor two in the gradient. Using half-MSE would halve that gradient, which matters when comparing learning rates across implementations. For binary classification, let $y_i \in \{0, 1\}$ and let $p_i = \sigma(z_i)$ be the predicted probability of class one. The probability assigned to the observed label is $p_i^{y_i}(1-p_i)^{1-y_i}$. Taking the negative mean logarithm gives binary cross-entropy (BCE). Minimising it increases the probability assigned to the observed labels, making it suitable for probabilistic binary classification:

$$C_{\text{BCE}} = -\frac{1}{m}\sum_{i=1}^{m}[y_i \ln p_i + (1-y_i)\ln(1-p_i)], \tag{6}$$

$$\frac{\partial C_{\text{BCE}}}{\partial p_i} = \frac{p_i - y_i}{m p_i(1-p_i)}, \tag{7}$$

$$\frac{\partial p_i}{\partial z_i} = p_i(1-p_i), \tag{8}$$

$$\frac{\partial C_{\text{BCE}}}{\partial z_i} = \frac{p_i - y_i}{m}. \tag{9}$$

The sigmoid derivative cancels when it is combined with this loss. For sigmoid output with MSE, the output gradient instead contains the factor $p_i(1-p_i)$, which becomes small at saturated outputs (Hjorth-Jensen 2026, Secs. 8.7–8.8).

For K mutually exclusive classes, a one-hot target Y_{ik} is one for the true class and zero elsewhere. Softmax turns the final preactivations, called logits here, into probabilities (Hjorth-Jensen 2026, Ch. 5, More than two classes: the softmax):

$$P_{ik} = \frac{e^{Z^L_{ik}}}{\sum_{r=1}^{K} e^{Z^L_{ir}}}, \qquad C_{\text{CE}} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} Y_{ik}\ln P_{ik}. \tag{10}$$

The probabilities sum to one for each observation. Softmax links them, so its derivative includes changes in the denominator. With $d_{kj} = 1$ when $k = j$ and zero

otherwise, the chain rule gives

$$\frac{\partial P_{ik}}{\partial Z_{ij}^L} = P_{ik}(d_{kj} - P_{ij}), \tag{11}$$

$$\frac{\partial C_{\text{CE}}}{\partial Z_{ij}^L} = -\frac{1}{m}\sum_k \frac{Y_{ik}}{P_{ik}} P_{ik}(d_{kj} - P_{ij}) \tag{12}$$

$$= -\frac{1}{m}\left(Y_{ij} - P_{ij}\sum_k Y_{ik}\right) = \frac{P_{ij} - Y_{ij}}{m}. \tag{13}$$

The final equality uses $\sum_k Y_{ik} = 1$ for each one-hot target. Thus both matched cross-entropy pairs give a probability-minus-target output error divided by batch size. This CE definition averages over observations, while MSE in Eq. (5) averages over all mK entries and also carries a factor two. These factors set the gradient scale.

To evaluate CE stably, each observation's loss is computed from logits as

$$\ln\left(\sum_k e^{Z_{ik}^L}\right) - \sum_k Y_{ik} Z_{ik}^L.$$

For the log-sum-exp term, subtracting the largest logit before taking exponentials and adding it back afterwards avoids overflow and taking the logarithm of an underflowed probability (Hjorth-Jensen 2026, Sec. 8.11) (Goodfellow et al. 2016, Sec. 4.1).

2.4 Backpropagation from the chain rule

Training requires gradients of the loss with respect to the network's trainable parameters. Backpropagation computes them efficiently by applying the chain rule backwards through the stored layer calculations. Define each loss derivative with respect to a preactivation as

$$\delta_{ij}^l = \frac{\partial C}{\partial Z_{ij}^l}.$$

The matched output–loss pairs give

$$\begin{aligned} \delta^L &= \frac{2}{mK}(A^L - Y) && \text{(linear MSE)}, \\ \delta^L &= \frac{p - y}{m} && \text{(sigmoid BCE)}, \\ \delta^L &= \frac{P - Y}{m} && \text{(softmax CE)}. \end{aligned} \tag{14}$$

For sigmoid BCE, p and y collect the binary probabilities and labels. A hidden activation feeds every neuron in the next layer, so the chain rule sums their contributions:

$$\frac{\partial C}{\partial A_{ij}^l} = \sum_k \delta_{ik}^{l+1} W_{jk}^{l+1}.$$

Multiplication by its activation derivative gives

$$\delta^l = \left(\delta^{l+1}(W^{l+1})^T\right) \odot f_l'(Z^l), \tag{15}$$

Since $\partial Z_{ik}^l / \partial W_{jk}^l = A_{ij}^{l-1}$ and a bias adds equally to every row, parameter gradients are

$$\frac{\partial C}{\partial W^l} = (A^{l-1})^T \delta^l, \qquad \frac{\partial C}{\partial b^l} = \sum_{i=1}^m \delta_{i,:}^l. \tag{16}$$

Here $\odot$ is elementwise multiplication. Each backward derivative is computed once and reused for both the parameter gradients and the preceding layer. This is reverse-mode automatic differentiation, with one forward/backward pass calculating all parameter gradients at a computational cost of the same order as a forward pass. This cost grows with network size (Hjorth-Jensen 2026, Sec. 8.8). The factors in δ^L carry the loss normalisation through the backward sweep.

2.5 Gradient-based training

Once gradients are available, an optimiser uses them to adjust the model parameters to reduce the loss. Let θ collect the trainable model parameters: polynomial coefficients or network weights and biases. Let $t = 1, 2, \ldots$ index updates and g_t be the gradient estimate of the objective being minimised at θ_{t-1} using the current batch. The objective includes penalty terms when regularisation is applied. Gradient descent (GD) takes the step

$$\theta_t = \theta_{t-1} - \gamma g_t.$$

The learning rate γ controls step size. Full-batch GD computes each gradient using the entire training set. Minibatches reduce the computational cost per update and change the gradient estimate between batches. An epoch is one pass through the training set, so smaller batches give more updates per epoch.

RMSProp rescales each coordinate using a running average of squared gradients. Adam also tracks a running first moment and corrects both moments for their zero initial values. These rules depend on the parameters and their gradients, so the same optimiser can train either task once backpropagation supplies the appropriate gradient (Hjorth-Jensen 2026, Secs. 4.7–4.11).

The shape of the objective affects what successful optimisation means. The linear-regression objectives are convex, so every local minimum is global. A nonlinear network cost is generally nonconvex, and its initial weights and learning rate can affect the fit reached. Completing a fixed number of epochs therefore leaves convergence to be checked separately (Hjorth-Jensen 2026, Ch. 4, convexity and nonconvex optimisation).

2.6 Scaling and initialisation

Input scaling and parameter initialisation influence the numerical behaviour and effectiveness of network training. The backward recursion repeatedly multiplies the weight matrices and activation derivatives defined above. The scale of these products can shrink gradients or make updates unstable (Hjorth-Jensen 2026, Secs. 8.9–8.10). Feature standardisation takes the form

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

Here μ_j and s_j are the mean and standard deviation of feature j, estimated from training data. Applying the same transformation to held-out data keeps their values out of the fitted preprocessing. Feature scaling affects preactivation sizes and the cost landscape

(Hjorth-Jensen 2026, Ch. 3, Scaling, centring and the intercept).

Initialisation sets the weights that first act on these inputs. Random weights break neuron symmetry: neurons with identical parameters can receive identical updates and fail to learn distinct features. Xavier and He schemes aim to limit changes in the scale of activations and gradients under simplifying assumptions. Their normal-distribution variances are $2/(n_{\text{in}} + n_{\text{out}})$ and $2/n_{\text{in}}$ respectively, where n_{in} and n_{out} count incoming and outgoing connections for a layer. Their effect depends on the activation function, so comparisons that change both assess their combined influence (Hjorth-Jensen 2026, Ch. 8, Weight initialisation).

2.7 Capacity and regularisation

Network capacity describes the range and complexity of functions a model can represent. In a neural network, increasing the number of hidden layers or neurons generally increases its capacity, allowing it to fit more complex patterns in the data. Polynomial degree likewise controls the complexity of the fixed-feature model. Falling training error with rising validation error is evidence of overfitting. The difference between validation and training error also depends on the split and training progress (Goodfellow et al. 2016, Ch. 5, Sec. 5.2). Weight-only regularised objectives are

$$J_1 = C + \lambda \sum_l \sum_{ij} |W_{ij}^l|, \tag{17}$$

$$J_2 = C + \lambda \sum_l \|W^l\|_F^2. \tag{18}$$

Here C is the task's data loss and $\lambda \geq 0$ sets the penalty strength. The objectives add a weight penalty to the data loss while leaving biases unpenalised. The squared Frobenius norm sums squared entries. L2 adds $2\lambda W^l$ to weight gradients. L1 adds $\lambda \operatorname{sign}(W^l)$ away from zero, with subgradient interval $\lambda[-1, 1]$ at zero. The value used at zero must belong to that interval (Hjorth-Jensen 2026, Secs. 3.9 and 8.12). L2 discourages large weights. L1 favours small or sparse weights (Goodfellow et al. 2016, Ch. 7, Sec. 7.1).

The polynomial regression models use the same distinction between data loss and penalty. For n observations, OLS minimises $\|X\theta - y\|_2^2/n$, with solution X^+y, where X^+ is the Moore–Penrose pseudoinverse. Ridge adds $\lambda\|\theta\|_2^2$, and Lasso adds $\lambda\|\theta\|_1$ (Hjorth-Jensen 2026, Ch. 3, least squares and penalties).

Early stopping limits fitting through the training duration. After each epoch, a held-out monitoring set provides the loss used to track improvement. The patience setting specifies how many epochs without improvement are allowed before stopping, and the fit restores the weights from the best monitoring epoch. This can act as regularisation, with its success depending on the stopping rule and monitoring data (Goodfellow et al. 2016, Sec. 7.8, Algorithm 7.1) (Nielsen 2015, Ch. 3, Early stopping).

2.8 Held-out assessment and model comparisons

The data splits give fitting and assessment distinct roles. Training data fit the parameters, validation data guide hyperparameter selection, and test data assess the fixed models (Hjorth-Jensen 2026, Ch. 3, cross-validation). Cross-validation rotates the validation fold within the development set, with preprocessing fitted separately on each training portion.

Repeatedly picking hyperparameters on validation scores can make the best reported value optimistic. A test set kept out of those choices provides a separate assessment of the fixed model (Goodfellow et al. 2016, Ch. 5, Sec. 5.3).

The training loss and final assessment metric can differ. Here we report regression MSE and assess classification with accuracy, CE and macro-F1. The predicted class has the highest softmax probability. Accuracy is the fraction of correctly predicted classes, while CE assesses the predicted probabilities using the loss defined in Section 2.3 (Hjorth-Jensen 2026, Ch. 5, likelihood and assessment). For confusion counts N, rows are true classes and columns predictions. Define

$$T_k = N_{kk}, \qquad F_k^+ = \sum_{i \neq k} N_{ik}, \qquad F_k^- = \sum_{j \neq k} N_{kj}.$$

These are class-k true positives, false positives and false negatives. The corresponding measures are

$$\begin{aligned} \text{precision}_k &= \frac{T_k}{T_k + F_k^+}, \\ \text{recall}_k &= \frac{T_k}{T_k + F_k^-}, \\ F1_k &= \frac{2T_k}{2T_k + F_k^+ + F_k^-}. \end{aligned} \tag{19}$$

Precision is the proportion of predictions for a class that are correct. Recall is the proportion of observations truly in that class that the model identifies correctly. F1 is their harmonic mean. Macro-F1 averages class F1 equally, giving rare classes the same weight as common ones (scikit-learn developers 2026a). Confusion counts identify which wrong labels remain (Hjorth-Jensen 2026, Sec. 5.12).

Multiclass logistic regression uses softmax with affine input logits and no hidden layer. It therefore provides a baseline for assessing the predictive value of nonlinear hidden features (Hjorth-Jensen 2026, Ch. 5, logistic regression and softmax).

3 Methods

3.1 Data and preprocessing

For the regression experiments, we retained the Runge dataset and split seeds and split proportions from Project 1. The data comprised 1000 evenly spaced inputs on $[-1, 1]$, with targets $y_i = (1 + 25x_i^2)^{-1} + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, 0.1^2)$. Data and split seeds were 42.

A shuffled split left 800 development points and 200 points reserved for testing. We reused the Project 1 test set. The retained polynomial settings and Project 2 network setups were selected using validation scores. Five shuffled folds within development gave 640 training and 160 validation points per split, with the same data split used for the model comparisons.

Following Section 2.6, feature means and standard deviations were estimated separately within each training portion. We centred targets with the training mean and added it back to predictions. For the Runge task, each network received the standardised value of x as its only input feature. Polynomial models received the separately standardised powers $x, \ldots, x^p$, without a constant column because the target mean supplied the intercept. Final fits recomputed these statistics on all development points.

Of the 205,797 letter images in AL-PUB v2, we excluded 21 blank images with conflicting labels, leaving 205,776 images across 24 classes. The dataset background and the v2 lunate-sigma change are described by Swindall et al. (2021) and Swindall (2024, Sec. 7.3). We first composited each image onto white and converted it to grayscale, then applied autocontrast separately to each image to stretch its grayscale intensity range. We resized it to fit within 26×26 pixels while keeping its aspect ratio, then centred it on a white 28×28 canvas. We flattened the canvas row by row and mapped pixel intensity p to $1 - p/255$, giving 784 input features in $[0, 1]$. There was no augmentation or class weighting. Figure 1 shows actual training crops alongside the prepared images and network intensities.

We kept images with the same POxy publication item number, identifying an individual entry in *The Oxyrhynchus Papyri* collection, in the same split because images from that item may share visual characteristics. This grouping aimed to reduce the risk of overly optimistic scores from related images appearing in both training and evaluation sets. Our project generated the holdout assignments during our initial AL-PUB data checks by shuffling publication groups with seed 2026 and reserving 20% of the groups, rounded upwards, for testing. The classification experiments reused these saved assignments. The split left 159053 development and 46723 test images, with 1135 and 284 publication groups respectively. Exact split ordering, preprocessing calls and saved image memberships are recorded in `supplementary/source/methods_reproducibility.tex`.

3.2 Network and training conventions

The NumPy FFNN followed Eq. (1) and the manual gradients in Eqs. (15)–(16). Runge used a linear output with full MSE from Eq. (5). Classification used 24 softmax outputs with mean CE and the output gradient in Eq. (13). CE was evaluated through log-sum-exp. The actual minibatch size set the gradient normalisation, including a short final batch.

Initialisation followed the activation-dependent schemes in Section 2.6: sigmoid hidden layers and output layers used Xavier normal weights, while ReLU and Leaky ReLU hidden layers used He normal weights. All biases started at zero. Leaky ReLU's negative slope was 0.01, and its assigned derivative at zero was also 0.01, compared with zero for ReLU. We used these conventions throughout float64 computation, including the gradient checks. Separate random generators controlled initialisation and shuffling, with the stated seed for each fit.

We used the optimiser rules in Section 2.5, with fixed decay rates and numerical-stability constants listed in Table 1 and constant base learning rates within each fit.

Optimiser	Decay rates	ε
RMSProp	$\rho = 0.9$	10^{-8}
Adam	$\beta_1 = 0.9$, $\beta_2 = 0.999$	10^{-8}

Table 1: Fixed decay rates and numerical-stability constants for the reused optimiser update rules.

We applied L1 and L2 penalties to weights only, following the supplied Week 44 implementation. Penalty gradients followed Section 2.7, without further sample or parameter division. For Adam, we added the penalty gradient to the loss gradient before updating its moments. The network L1 subgradient was zero at zero. The reused polynomial Lasso solver used $+1$ there.

To match our polynomial objectives, scikit-learn Ridge used $\alpha = n\lambda$ and Lasso used $\alpha = \lambda/2$, where n is the training size (scikit-learn developers 2026c, objective definitions). Predictive comparisons used unpenalised scores.

3.3 Runge comparisons

We compared sigmoid networks with hidden widths [50] and $[100, 100]$, using four learning rates from 0.001 to 0.1. Each fit ran for 1000 epochs, with full-batch GD and batches of 32 for RMSProp/Adam. We picked a learning rate for each architecture/optimiser pair by mean validation MSE, then refitted the six fixed setups on all development points for evaluation on the reserved test set. The OLS comparison kept the degree-14 setting from the Project 1 report (Bell-Mathiesen 2026).

We then tested whether added capacity improved held-out prediction or mainly improved the training fit. We compared three hidden activations with one or two layers of width 25, 50 or 100. Each architecture/activation had a separately selected learning rate from $\{0.0003, 0.001, 0.003\}$. Adam fits used 1000 epochs, batches of 32 and seed 2026. The capacity comparison examined the difference between validation and training MSE and produced no test scores. Activation changes also changed initialisation.

We compared unregularised networks with L1- and L2-regularised networks to test whether the penalties improved prediction. To limit computational cost, we used random search to evaluate 16 hyperparameter configurations (λ, γ, and network architecture) rather than an

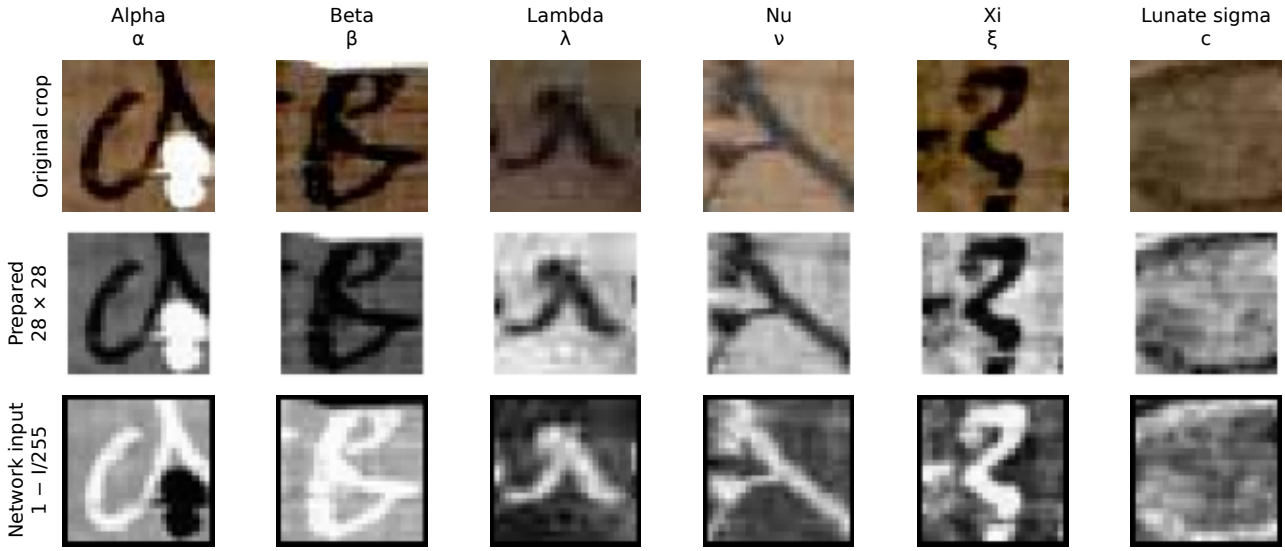


Figure 1: AL-PUB v2 training examples through preprocessing. One image per shown class was randomly sampled from the first cross-validation split's fitting portion, with sampling seed 2718. The middle row shows grayscale, autocontrast and aspect-preserving resizing on the white 28×28 canvas. The bottom row shows the inverted intensities supplied to the network, displayed spatially before flattening. Here I is a middle-row pixel value. Crops are enlarged independently for viewing. Several examples are difficult to make out: background texture and neighbouring ink strokes add visual noise that remains after preprocessing.

exhaustive grid search (Bergstra and Bengio 2012). The L1 and L2 comparisons used the same randomly drawn configurations. We picked the lowest mean five-fold validation MSE for each penalty choice at seed 2026, then fitted each picked setting on all development points using the same seed. Ridge degree 14 at $\lambda = 10^{-6}$ and Lasso degree 6 at $\lambda = 10^{-4}$ were picked using Project 1's five-fold validation results (Bell-Mathiesen 2026). Lasso selection included only candidates marked converged in every fold. We refitted the polynomial models with these fixed degrees and penalty strengths on the shared Project 2 data. Appendix A gives the search ranges, sampling procedure and solver checks.

3.4 Letter classification

We used a stratified 24000-image development sample to reduce the time needed for the search. Five validation folds used the publication grouping described in Section 3.1. Within each fold's training portion, a separate publication-grouped monitoring set controlled early stopping, while validation macro-F1 ranked setups. The monitoring set determined when to stop each training run, while the validation fold was kept separate to compare model configurations. Training stopped after ten epochs without a decrease in monitoring cross-entropy or after 100 epochs, restoring the weights with the lowest monitoring loss. All fits used batches of 128 and seed 2026. For the minibatch workflow, we consulted the public Chapter 11 code accompanying Raschka et al. (2022b) (Raschka et al. 2022a). We used our own grouped monitoring procedure.

The manual search tested twelve setups that varied architecture and activation, with no penalty or an L1/L2 penalty. Logistic regression had no hidden layer or penalty and tested learning rates $\{10^{-4}, 10^{-3}, 10^{-2}\}$. We picked setups by mean validation macro-F1 (the unweighted average of the 24 class-specific F1 scores), assigning zero to undefined class F1 scores (scikit-learn developers 2026a). The library FFNN matched the best sampled unpenalised sigmoid/ReLU setting and grouped stopping rule, while keeping its own initialisation and Adam details (scikit-learn developers 2026b, MLPClassifier and partial_fit). Appendix A gives the full ranges.

The final models trained on all 159,053 development images. We adjusted the number of training epochs to maintain approximately the same total training exposure as during model selection (Appendix A). All settings were fixed before evaluating the models on the test set.

Classification training curves average cross-entropy values evaluated as the weights change throughout each epoch, whereas the validation tables report scores using the weights from the epoch with the lowest monitoring loss.

3.5 Checks and uncertainty

We checked gradients against independent float64 JAX losses, including penalties, zero conventions and short minibatches. Reference derivatives used the same assigned values at nondifferentiable points (Hjorth-Jensen 2026, Ch. 4, automatic differentiation). We checked that a linear network could learn a simple regression by comparing its predictions with the direct OLS solution. We also checked the polynomial regression fits with independent solvers. We loaded saved model weights and checked that they reproduced the reported predictions. For classification, we checked that publication groups

stayed separate across the training, monitoring and validation sets, and that reserved test data were excluded from those sets. Pass thresholds are in Appendix A.3.

We report the standard deviation of scores across the five validation folds to describe how much their scores varied. These runs used training seed 2026, so this measures variation between folds rather than across training seeds. The assessment limits are discussed in Section 5.4.

3.6 Software

NumPy supplied array calculations (Harris et al. 2020), and SciPy supplied stable numerical functions and sampled distributions (Virtanen et al. 2020). Scikit-learn supplied the data splitters and hyperparameter sampling, alongside the library model and reference solvers (Pedregosa et al. 2011). JAX supplied independent reference gradients (Bradbury et al. 2018). Tables used pandas (McKinney 2010). Report figures used Matplotlib (Hunter 2007). Notebook regression heatmaps used seaborn (Waskom 2021). Package versions are in `requirements-lock.txt`.

4 Results

Runge test scores use the 200 reserved test points. Letter test scores use the 46723 reserved test images after full-development fits.

4.1 Implementation checks

Manual gradients agreed with independent float64 JAX references. Appendix A.3 gives the recorded errors and gradient-check thresholds.

The network reproduced the direct OLS predictions in the simple regression check. Cross-entropy evaluation remained stable at extreme logits in the test cases. Penalty checks confirmed bias exclusion and recovery of the unpenalised computation at zero penalty. Early stopping restored the lowest-monitoring-loss weights, and reloaded weights reproduced the reported predictions. Publication groups stayed separate across the classification data partitions, and reserved test data were excluded from training and validation. These outcomes support the tested assessment path, with saved weights and fold scores linked from Appendix A.

4.2 How training affects the Runge comparison

Following the implementation checks (Section 4.1), we picked the network for comparison with OLS by mean validation MSE within the optimiser comparison (Table 2). The subsequent capacity and regularisation searches were separate experiments and did not change this selection.

Adam gave the lowest validation MSE for both sigmoid architectures at 1000 epochs (Table 2). GD's training and validation errors both stayed high. Full-batch gradient descent made one update per epoch on each training fold, while the minibatch methods made 20.

Hidden widths	Optimiser	Learning rate	Training MSE	Mean validation MSE	Fold SD
[50]	GD	0.1	0.078119	0.078699	0.011378
[50]	RMSProp	0.001	0.010097	0.010417	0.000940
[50]	Adam	0.001	0.009829	0.009979	0.000768
[100, 100]	GD	0.1	0.092019	0.092468	0.010901
[100, 100]	RMSProp	0.001	0.009745	0.010129	0.000985
[100, 100]	Adam	0.001	0.009672	0.009896	0.000799

Table 2: Validation-selected learning rate for each sigmoid architecture and optimiser. Scores use the five shared Runge folds, seed 2026 and 1000 epochs.

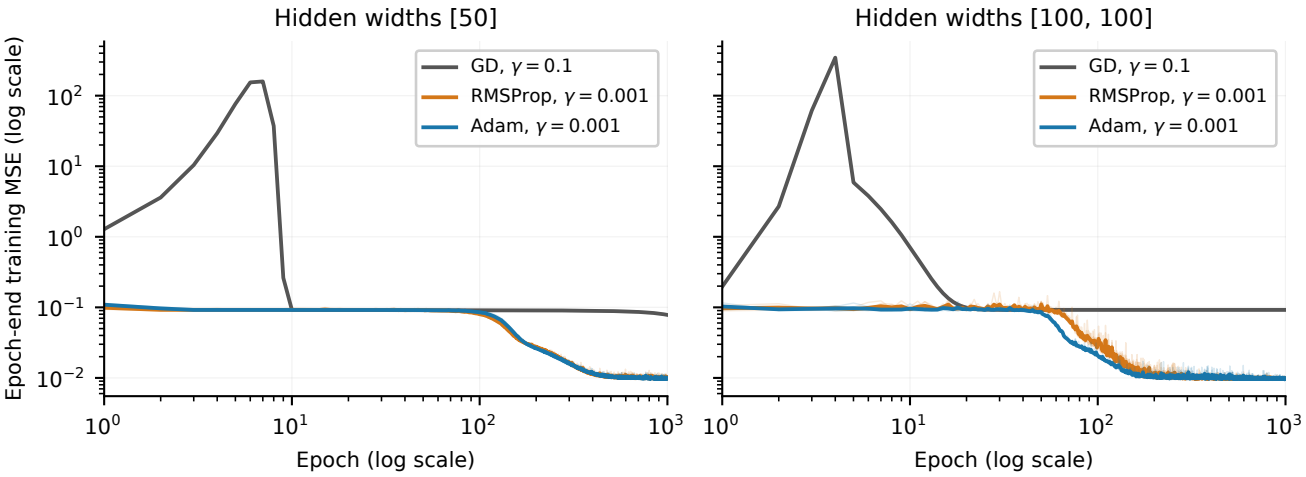


Figure 2: Epoch-end training MSE for the validation-picked GD, RMSProp and Adam settings in each sigmoid architecture. Thin lines show five fitting folds and bold lines their mean. Both axes are logarithmic. GD used one full-batch update per epoch, while the minibatch methods used 20.

Figure 2 shows GD's early loss excursions and persistently high training MSE. Adam and RMSProp reached much lower errors, with their two-layer fits improving earlier. This helps explain the final-score differences. These curves compare the tested training procedures, including their unequal batch sizes and update counts per epoch.

Learning-rate sensitivity differed between the tested network setups. With RMSProp, both networks achieved a validation MSE of approximately 0.010 at a learning rate of 0.001. Increasing the learning rate to 0.1 raised the validation MSE to 0.0143 for the one-layer network and 0.0552 for the two-layer network (Appendix B, Table 10).

4.2.1 Can learned features improve on polynomial regression?

On the reserved Runge test data, the validation-selected two-layer Adam network and degree-14 OLS achieved similar test MSE (Table 3). Both models captured the central peak (Figure 3), suggesting that the fixed polynomial features were sufficient to achieve comparable predictive performance on this dataset. The lower panel shows where the network predicts above or below OLS. Although some alternative network setups achieved lower test MSE, the main comparison retained the network picked by validation performance within the optimiser comparison, without using test results to change that selection.

Hidden widths / model	Training MSE	Test MSE
[100, 100] Adam (validation pick)	0.010312	0.011367
[100, 100] GD	0.092064	0.083562
[100, 100] RMSProp	0.010157	0.010870
[50] Adam	0.010051	0.011147
[50] GD	0.078250	0.070818
[50] RMSProp	0.011636	0.012513
OLS degree 14	0.009890	0.011217

Table 3: Fixed final Runge fits on 800 development points, scored on the 200 reserved test points.

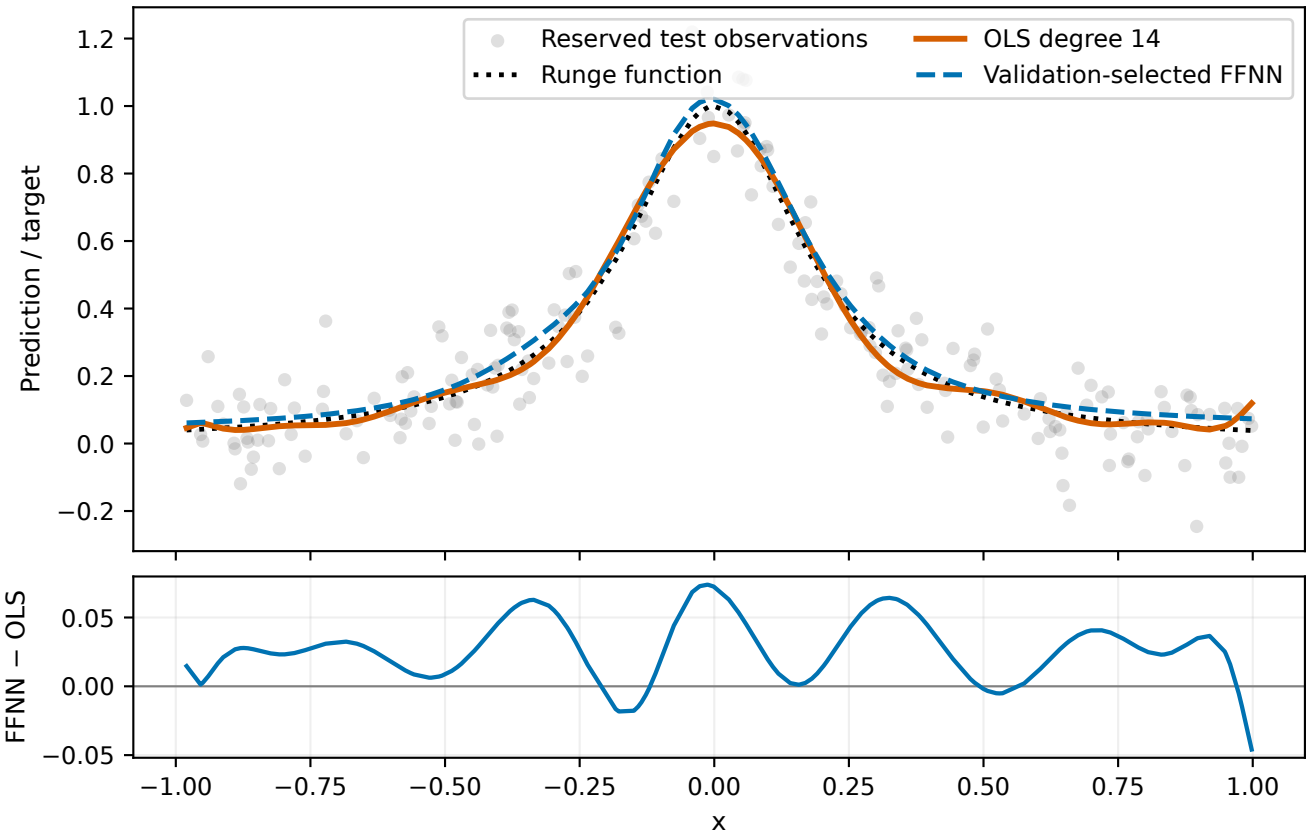


Figure 3: Predictions from the validation-selected FFNN and fixed degree-14 OLS fit on the 200 reserved Runge test points. Fits used 800 development points. Both model types followed the central peak. The lower panel shows FFNN predictions minus OLS predictions.

4.3 Does greater capacity improve generalisation?

Adam produced the lowest validation error in the training comparison, so we used it to examine how activation and network size affected generalisation.

The lowest validation error came from a small two-layer sigmoid network. For both ReLU and Leaky ReLU, the best fit instead used one narrow hidden layer (Table 4).

Activation	Hidden widths	Learning rate	Training MSE	Mean validation MSE	Fold SD
Leaky ReLU	[25]	0.0003	0.009680	0.009999	0.000512
ReLU	[25]	0.0003	0.009668	0.010000	0.000517
Sigmoid	[25, 25]	0.0003	0.009580	0.009787	0.000739

Table 4: Lowest mean validation MSE per activation in the width/depth grid. Learning rates were picked on the same five folds. All fits used Adam, seed 2026 and 1000 epochs.

For two-layer ReLU networks, raising width from 25 to 100 lowered training MSE from 0.009549 to 0.009288 while raising validation MSE from 0.010259 to 0.010609, with worse validation scores in all five folds. The gap between validation and training MSE grew from 0.000710 to 0.001321. Leaky ReLU followed the same direction (Figure 4). Falling training error with rising validation error supports overfitting for these architectures and learning rates.

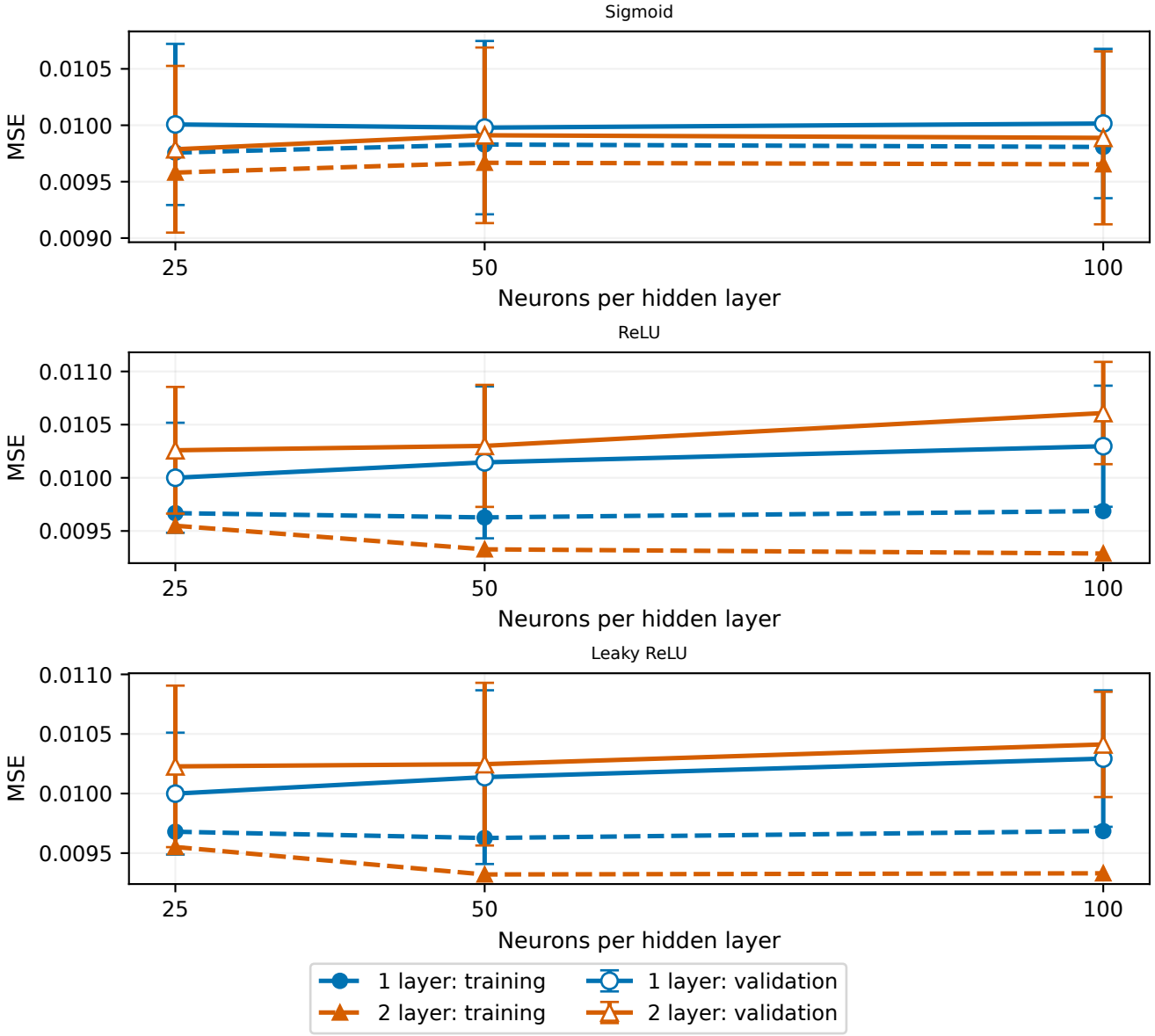


Figure 4: Training and validation MSE across width and depth. Each point uses its validation-selected learning rate, Adam, matched hidden-layer initialisation and 1000 epochs. Validation error bars show Fold SD. The output is linear and MSE has no penalty.

The sigmoid networks showed a different pattern: at width 25, adding a second layer improved both training and validation MSE, while widening the two-layer network to 100 slightly worsened both. Together with the ReLU and Leaky ReLU results, this shows that more capacity did not have a uniform benefit for the tested architectures and learning rates. The validation error bars overlap substantially, showing that the differences between setups are small relative to the variation between folds.

Each activation/architecture pair had its own picked learning rate, and changing activation also changed initialisation as described in Section 3.2, so the capacity comparison does not isolate these effects. The complete learning-rate grid is in Appendix B (Table 11).

4.4 Do penalties improve prediction?

The larger ReLU and Leaky ReLU networks showed that extra capacity could worsen generalisation. This raises the question of whether weight penalties improve prediction. The penalty search used sigmoid networks with separately selected hyperparameters. Therefore, the comparison does not isolate the effect of regularization or establish whether it would reduce the overfitting observed in the ReLU networks.

The validation-selected penalised and unpenalised networks had nearly identical validation errors (Table 5). Figure 5 compares L1 and L2 with architecture, learning rate and penalty strength held fixed within each connected pair. The selected networks in Table 5 used small penalties. The sampling inventory is preserved in `supplementary/figures/penalty_search_inventory.pdf`.

Model	Hidden widths	Learning rate	λ	Mean validation MSE	Fold SD
Unregularised FFNN	[50, 50]	0.000465	0	0.009929	0.000756
L1 FFNN	[25, 25]	0.00871	1.15e-06	0.009899	0.000911
L2 FFNN	[25, 25]	0.00871	1.15e-06	0.009871	0.000859

Table 5: Architectures, learning rates and penalty strengths picked by mean unpenalised validation MSE across five folds at seed 2026. Fold SD describes variation across those folds.

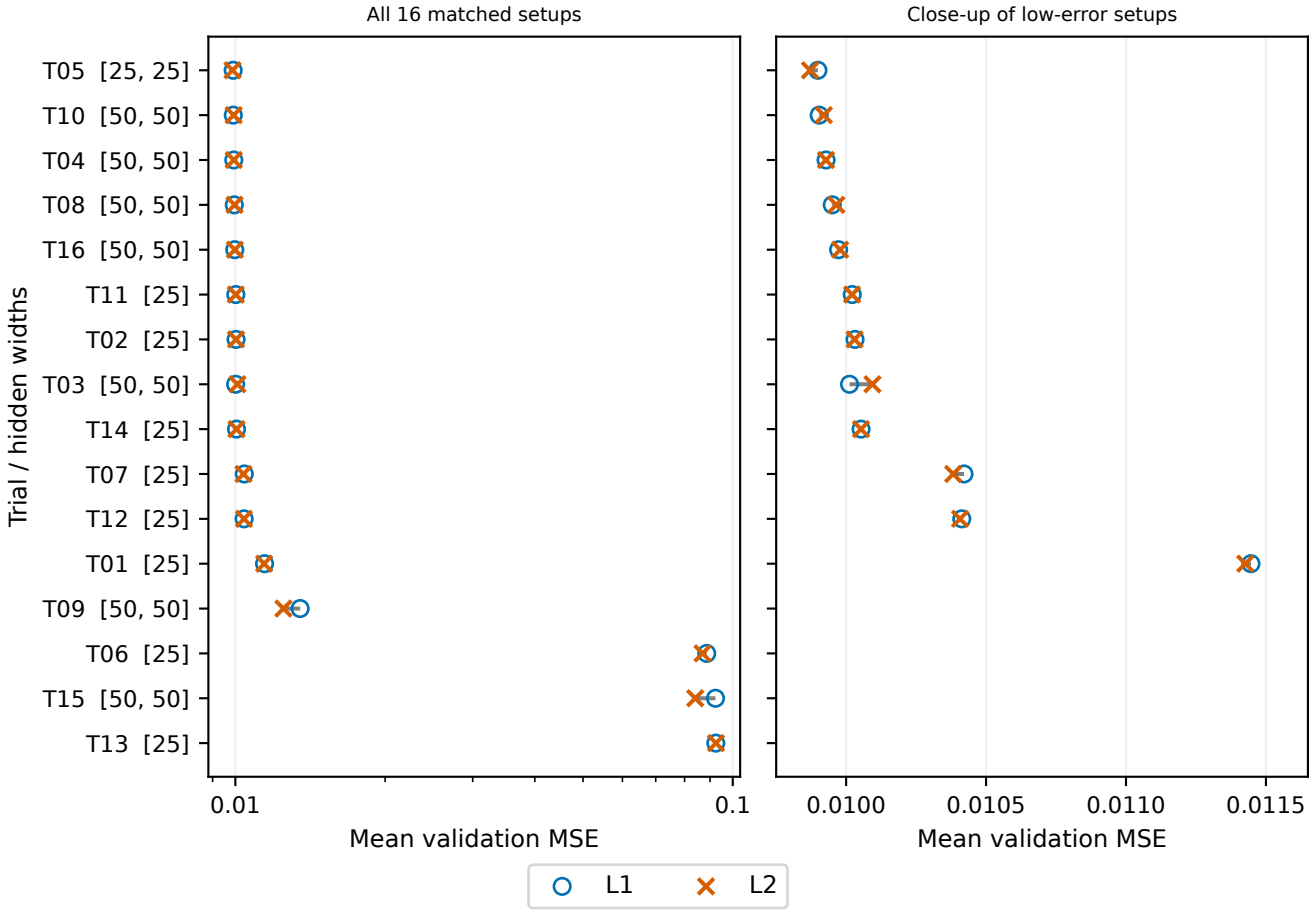


Figure 5: Matched L1/L2 mean validation MSE across the same five folds at seed 2026. Each row connects scores for one architecture, learning rate and penalty strength, sorted by the mean of its L1 and L2 scores. Trial labels use one-based numbering and show hidden widths. The left panel includes all 16 pairs on a logarithmic axis. The right panel enlarges the low-error range and omits points outside it. The exact values are in the supplementary matched-setup CSV listed in Appendix A.

Figure 5 shows that L1 and L2 gave nearly identical validation errors for most low-error setups, with neither penalty consistently better. A few setups fitted poorly with both penalties. Overall, the sampled architectures, learning rates and penalty strengths affected validation error more than the choice between L1 and L2.

At seed 2026, test MSE was 0.010810 without a penalty, 0.010931 with L1 and 0.010805 with L2 (Table 6). These small differences give little evidence of a useful predictive gain from penalising the network in this experiment.

The penalised and unpenalised neural curves almost coincide (Figure 6). Lasso noticeably underestimates the central peak, while Ridge shows a smaller shortfall there. These departures help explain their higher test errors in Table 6. The Ridge and Lasso comparison applies to the fixed Project 1 degrees and penalty strengths, which were not retuned here.

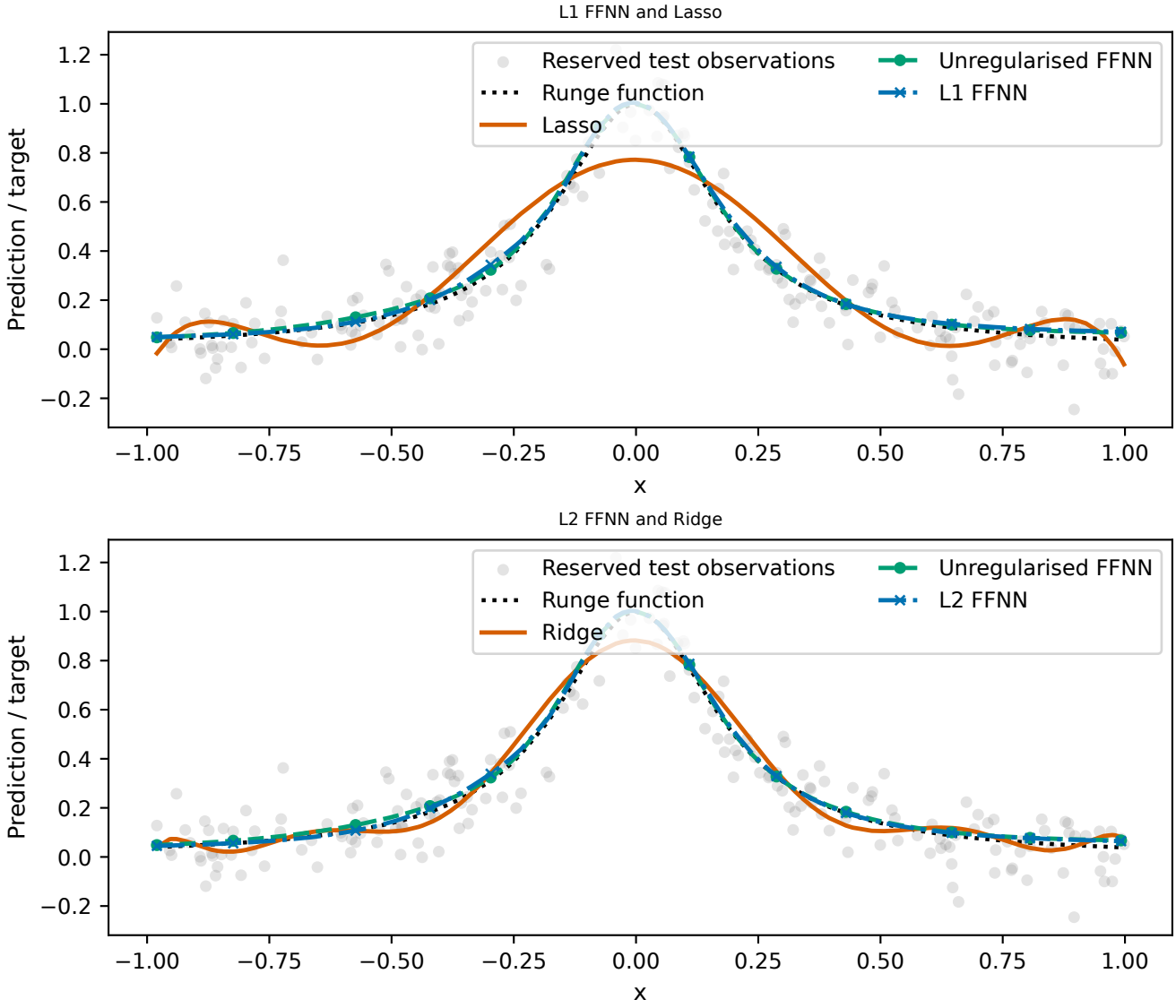


Figure 6: Fixed penalised and unpenalised FFNNs versus the fixed Project 1 polynomial regression models on the same test points. Neural curves and scores use one final fit at seed 2026, with architecture, learning rate and penalty strength fixed before test assessment.

Model	Test MSE
Unregularised FFNN	0.010810
L1 FFNN	0.010931
L2 FFNN	0.010805
Ridge	0.013428
OLS degree 14	0.011217
Lasso	0.018774

Table 6: Test MSE of the selected networks at seed 2026 and the fixed polynomial regression models on the 200 reserved test points. Each row reports one final fit.

4.5 Do nonlinear features improve letter classification?

We next investigated whether learned nonlinear features provide an advantage in handwritten letter classification, using logistic regression as a linear baseline. Both models used pixel inputs, while the FFNN could learn nonlinear combinations of them.

Model selection used five publication-grouped validation folds from a 24,000-image subset of the development data (Table 7). The picked models were then fitted on the full development set and evaluated on the 46,723 reserved test images (Table 9).

The picked manual FFNN used one 128-neuron ReLU hidden layer without a penalty. It outscored logistic regression on validation and test images. Table 8 gives the hidden widths, learning rates and final number of epochs for the full-development fits.

Model	Mean validation macro-F1	Macro-F1 Fold SD	Mean validation accuracy	Mean validation CE
Manual FFNN	0.4635	0.0097	0.5598	1.6263
Logistic regression	0.3880	0.0099	0.4931	1.9060
Scikit-learn FFNN	0.4650	0.0147	0.5652	1.6304

Table 7: Scores on five grouped validation folds of the 24000-image search subset. Macro-F1 Fold SD is the sample standard deviation across folds at seed 2026.

Model	Hidden widths	Learning rate	Final epochs
Manual FFNN	[128]	0.000623163	7
Logistic regression	None	0.001	3
Scikit-learn FFNN	[128]	0.000623163	7

Table 8: Hidden widths, learning rates and final number of epochs for the full-development fits. Both FFNNs had ReLU and no penalty. The number of epochs was set from the best monitoring epochs on development data.

The search tested one to three hidden layers with widths 32, 64 or 128 and sigmoid, ReLU or Leaky ReLU activations. Added depth and penalties therefore did not produce the best sampled fit. Architecture, activation and learning rate changed together, so their individual effects cannot be separated. Manual validation macro-F1 ranged from 0.0616 to 0.4635, showing that using a hidden layer alone did not ensure a useful classifier. The full search inventory is preserved in `supplementary/figures/classification_search.pdf`.

Logistic regression gives a clearer comparison of learning rates because its architecture stayed fixed. Learning rates 10^{-4}, 10^{-3} and 10^{-2} gave validation macro-F1 of 0.3691, 0.3880 and 0.3628, respectively. The middle learning rate was best under the common stopping procedure. The lowest learning rate reached the epoch ceiling in every fold, so its lower score may reflect incomplete fitting within 100 epochs.

The picked manual model's training macro-F1 exceeded its validation macro-F1 by 0.1150. The highest macro-F1 and lowest CE came from different setups, so the loss used to train the network and the measure used to pick it did not rank every fit alike. Early stopping under the rule in Section 3.4 was triggered in every cross-validation fold for each validation-selected model. The saved fold scores and stopping epochs are listed in the supplementary instructions for reproducing the results linked from Appendix A.

On test, the nonlinear manual FFNN exceeded the linear logistic-regression baseline by 0.1214 macro-F1 and 11.06 percentage points in accuracy under the tested training configurations (Table 9). The library FFNN scored slightly higher on validation and lower on test, with one training seed for each final model. The number of tested setups and final number of epochs differed between the model families.

Always predicting Omicron, the most frequent class in the full-development training data, gave a majority-class baseline accuracy of 10.86% on test. Counts and calculations are saved in `results/classification_part_e/image_independent_baselines.json`.

Model	Test macro-F1	Test accuracy	Test CE
Manual FFNN	0.5086	0.6092	1.4579
Logistic regression	0.3873	0.4986	1.8554
Scikit-learn FFNN	0.4926	0.6034	1.4759

Table 9: Final model scores on the same 46723 reserved test images.

Figure 7 shows large differences in recognition between letters. Several rows have a strong diagonal, while others spread across several predicted letters. Lambda was labelled Alpha much more often than Alpha was labelled Lambda.

Only seven of 163 Xi images were correctly identified, giving recall 0.0429 despite precision 0.7778 (Figure 8). The network therefore seldom predicted Xi and missed most true Xi images. For potential transcription assistance, an overall improvement would still leave that letter poorly recovered.

Figure 8 also shows that the development data were highly uneven across letters. Xi had few training examples and very low recall, but class frequency alone does not explain performance. Lambda had many more examples and was still often misclassified. Visual overlap and limited training support may contribute to these errors. The full per-letter measures are saved in `results/classification_part_e/final/manual_per_class.csv`.

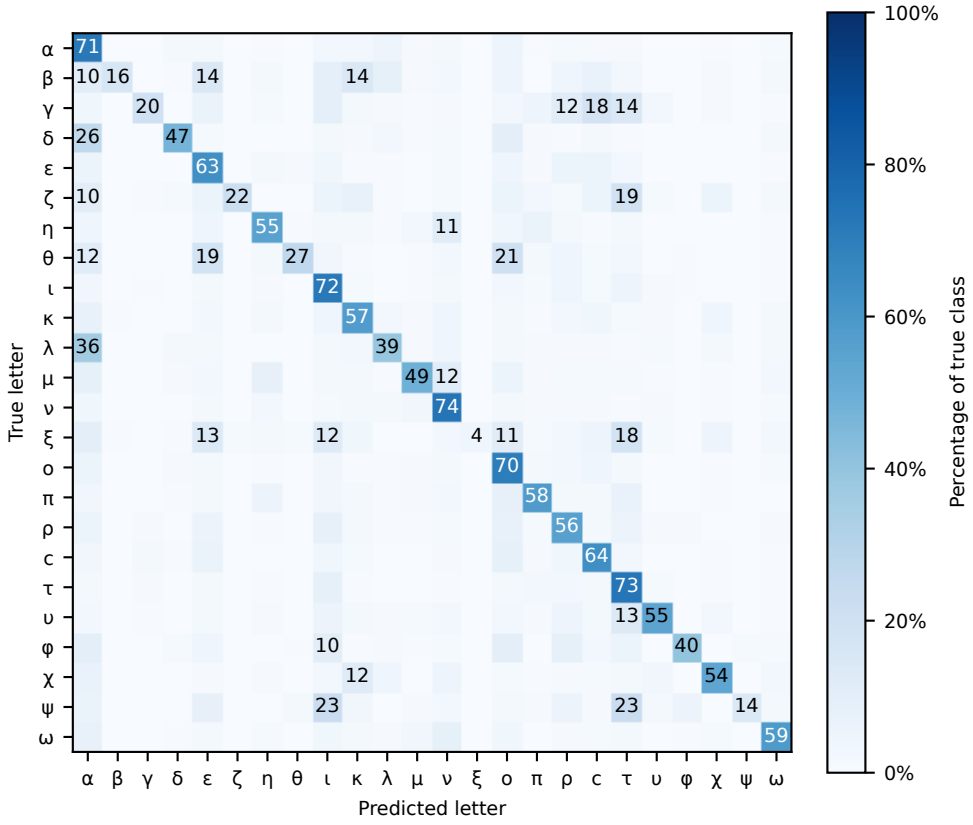


Figure 7: Row-normalised confusion matrix of the fixed manual FFNN on 46723 reserved test images. Rows are true letters and columns are predicted letters, in the saved 24-letter output order. Cell labels are percentages, shown on the diagonal and for off-diagonal shares of at least 10%.

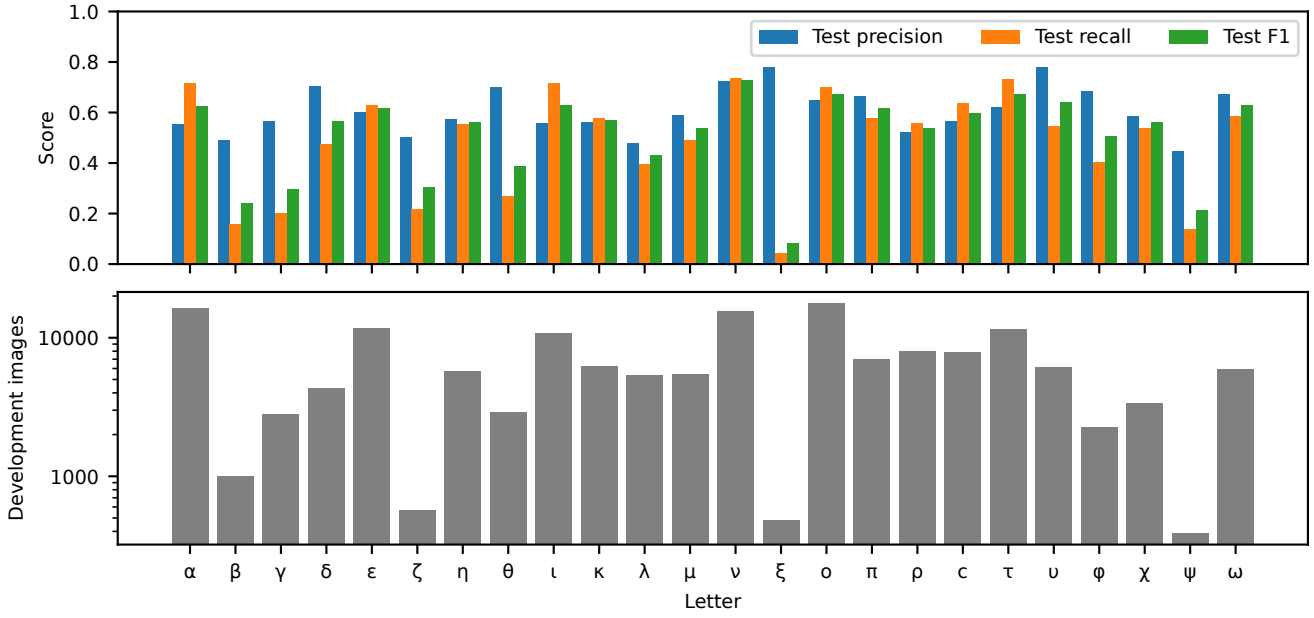


Figure 8: Manual-FFNN test precision, recall and F1 by letter, with development-set class counts on a logarithmic scale. Xi had very low recall. The sizeable Lambda-to-Alpha error in the confusion matrix also concerns a more frequent letter.

5 Discussion

5.1 Polynomial models and learned features

OLS remained a practical Runge model, directly fitting coefficients for predefined polynomial features. Neural networks gained flexibility by learning internal features through parameter optimisation, requiring architecture selection and iterative training. Greater network capacity could improve the training fit while worsening generalisation (Figure 4), so flexibility alone gave no consistent predictive advantage.

Ridge and Lasso constrain polynomial coefficients through an extra penalty parameter. Neither selected penalised polynomial model improved on OLS (Table 6).

For networks, the 16 matched L1/L2 comparisons held architecture, learning rate and penalty coefficient fixed within each pair (Figure 5). They isolate the penalty-form comparison at those architectures, learning rates and penalty strengths and gave no consistent L1/L2 preference. The pairwise ranking does not identify an optimal strength, since architecture and learning rate change between pairs. Similar scores for the separately picked penalised and unpenalised networks showed little gain within this search. A learning-rate/penalty grid at fixed architecture, including zero penalty, would test whether a useful strength emerges when learning rate is retuned (Section 2.7).

Letter classification showed a clearer benefit from learned features. Logistic regression's 49.86% test accuracy exceeded the majority-class baseline of 10.86% (Section 4.5), showing that the linear classifier extracted useful information from pixel values. Both FFNNs improved recognition, with similar scores despite their separate implementations (Tables 7 and 9). Their flexibility required more parameters and a broader search. The manual code exposes the forward and backward calculations, while the library reduces the custom training code to maintain.

5.2 Comparison with published character recognition

Swindall et al. (2021, Table IV) compared convolutional approaches on an earlier AL-PUB dataset. Their convolutional network (ResNet) achieved mean validation accuracy of 92.57% over ten folds, compared with our FFNN's test accuracy of 60.92%. Their release contained 195683 images, while we used v2 with a publication-grouped holdout and different preprocessing. Their paper does not specify whether its folds kept publication groups together. Its mean cross-validation accuracy and our reserved-test score therefore provide context rather than a matched benchmark.

A convolutional neural network (CNN) learns filters that act on local image regions and share weights across positions (Goodfellow et al. 2016, Ch. 9). This structure could help recognise strokes appearing in different parts of a letter image. Our dense network can learn spatial relationships, but its architecture does not impose this local weight sharing.

5.3 Training rules and learning rates

Under the tested training settings, Adam gave the lowest validation MSE for both sigmoid networks, and RMSProp also fitted Runge substantially better than plain GD (Table 2). Learning-rate sensitivity (Table 10) makes tuning part of that finding. GD's high training error shows that it fitted the data poorly within 1000 epochs. GD requires less optimiser state, while RMSProp and Adam store moving gradient moments (Section 2.5).

Full-batch GD made fewer updates per epoch than mini-batch RMSProp and Adam. This comparison leaves the contribution of adaptive updates unresolved. Holding batch size and update count fixed, with learning rates tuned separately for each method, would give a clearer comparison of the update rules.

5.4 Scope of the assessment

Network fits used one training seed, and convergence was unverified. Repeated seeds with checks of objective stability under extended training would show whether small ranking differences persist. Fold standard deviations describe variation in validation scores across folds whose training subsets overlap (Hjorth-Jensen 2026, Ch. 3, cross-validation). Publication grouping kept each publication's images together, but writer identities were unverified. The same writer could therefore appear in multiple publication groups and consequently in both training and test sets. Generalisation to unseen writers remains unsettled.

6 Conclusion

For Runge regression, the L2-regularised neural network selected by the penalty search achieved a test MSE of 0.010805, compared with 0.011217 for polynomial OLS, with little improvement over the unregularised network. OLS remains a useful practical choice because it achieves competitive predictive performance through a direct fit of fixed polynomial features, without requiring iterative neural-network training.

For letter classification, the selected manual FFNN substantially outperformed logistic regression on the reserved test set in both macro-F1 and accuracy. Separately, the library FFNN achieved slightly higher mean validation macro-F1 than the manual FFNN. The models differed in the number of tested setups, and architecture, activation and learning rate varied together. Architecture's own contribution remains unresolved. Low recall for the letter Xi indicates that many actual instances of this character are missed, a weakness for transcription assistance.

A useful next study would compare a CNN with the present dense FFNN using the same preprocessing and publication-grouped development folds, with the same

number of tested setups and number of epochs across several seeds. This would test whether convolutional structure improves recognition beyond the dense model. Per-letter recall would show whether any gain also helps poorly recovered letters such as Xi. Final assessment on previously unused publications with verified writer separation would test transfer to new source material.

A Experiment settings and verification

The main Methods describe the data and assessment procedure. The search ranges below complete the search specification. Detailed execution instructions and exact implementation details are kept in `supplementary/REPRODUCIBILITY.md` and `supplementary/source/`. Fold scores and stopping epochs remain under `results/classification_part_e/`. The repository URL and revision remain to be added before hand-in.

A.1 Search settings

All Runge searches used the five shared shuffled folds with seed 42, training seed 2026, 1000 epochs and batches of 32 for Adam/RMSProp. Full-batch GD used all training points per update. Its learning-rate grid, also used for the optimiser comparison's Adam/RMSProp fits, was $\{0.001, 0.01, 0.03, 0.1\}$. The capacity grid is given in Methods.

The regularisation search used sigmoid networks with hidden widths [25], [25, 25] or [50, 50]. Scikit-learn's `ParameterSampler(n_iter=16,random_state=42)` drew learning rates log-uniformly on $[10^{-4}, 10^{-2}]$ and positive penalty strengths log-uniformly on $[10^{-7}, 10^{-2}]$. L1 and L2 shared the draws, and the unregularised family used the same architectures and learning rates with zero penalty. Each family was ranked by mean unpenalised validation MSE. The exact sixteen rows are in `supplementary/tables/matched_penalty_settings.csv`.

Classification used a stratified 24000-image development sample with seed 2026 and five `StratifiedGroupKFold` validation splits with that seed. Within outer fold $f = 0, \ldots, 4$, the first split of another five-fold grouped splitter, with seed $2027 + f$, supplied the monitoring set. `ParameterSampler(n_iter=12,random_state=2026)` sampled one to three equal-width hidden layers, widths $\{32, 64, 128\}$, and sigmoid, ReLU or Leaky ReLU. Penalties were none, L1 or L2. Learning rates and positive penalty strengths used the same log-uniform ranges as above, with strength set to zero for no penalty. Mean validation macro-F1 ranked setups.

Final classification fits used all 159053 development images. Training exposure counts the total number of training observations processed across epochs, including repeated observations in successive epochs. For each fold, exposure was measured at the best monitoring epoch $E_{\mathrm{best},f}$, which can precede the epoch at which early stopping was triggered. Multiplying it by the fold fitting size $n_{\mathrm{fit},f}$ gave that fold's exposure. We used the median exposure across folds as the reference and divided by 159053 to obtain the final epoch count, rounding upwards and requiring at least one epoch. Using only development data, this gave

$$E_{\mathrm{final}} = \max\left(1, \left\lceil \frac{\mathrm{median}_f(E_{\mathrm{best},f} n_{\mathrm{fit},f})}{159053} \right\rceil\right).$$

The resulting epochs are reported beside the final classification results.

A.2 Polynomial solver checks

Ridge used the fixed degree-14, $\lambda = 10^{-6}$ setting with `numpy.linalg.solve` on the normal equations containing $n\lambda I$. It was checked against augmented least squares. The fixed degree-6, $\lambda = 10^{-4}$ Lasso used the reused Adam/JAX solver, learning rate 0.01, at most 10000 updates and stopping threshold 10^{-7} on the coefficient change norm. Initial coefficients used $\mathcal{N}(0, 0.1^2)$ with seed 42. The independent scikit-learn Lasso check used $\alpha = \lambda/2$, `fit_intercept=False`, `tol=1e-12` and `max_iter=100000`. Solver convergence flags and cost differences remain with the saved results.

A.3 Gradient checks

Each tested entry passed when $|g - g_{\mathrm{ref}}| \leq a + r|g_{\mathrm{ref}}|$. The core reuse checks used $a = 10^{-11}$ and $r = 10^{-9}$. The optimiser comparison has separate saved gradient fixtures. Its reported maximum absolute error was 4.44×10^{-16}.

Check set	a	r	Maximum absolute error
Capacity	10^{-10}	10^{-9}	8.88×10^{-16}
Regularisation	10^{-11}	10^{-9}	8.88×10^{-16}
Classification	2×10^{-12}	2×10^{-11}	6.94×10^{-17}

The tested cases and gradient errors are preserved with the supplementary check results. Separate checks cover loss values and stopping rules.

B Learning-rate sensitivity

Table 10 gives the complete learning-rate grid for the optimiser comparison in Section 4.2.

Hidden widths	Optimiser	$\gamma = 0.001$	0.01	0.03	0.1
[50]	GD	0.094381	0.092486	0.092420	**0.078699**
[50]	RMSProp	**0.010417**	0.010445	0.011110	0.014306
[50]	Adam	**0.009979**	0.010026	0.010311	0.011180
[100, 100]	GD	0.092558	0.092536	0.092533	**0.092468**
[100, 100]	RMSProp	**0.010129**	0.011976	0.010570	0.055178
[100, 100]	Adam	**0.009896**	0.010558	0.011918	0.011389

Table 10: Mean validation MSE across the four learning rates for each sigmoid architecture and optimiser. Bold entries mark the lowest mean in each row. Every setup used the same five folds, seed 2026 and 1000 epochs. GD used full batches and RMSProp/Adam used minibatches of 32.

The narrowest one-layer sigmoid network had high validation error at the smallest learning rate. Two-layer sigmoid scores changed less across the tested rates. The largest rate worsened wider one-layer ReLU and Leaky ReLU fits.

Table 11: Mean validation MSE across learning rates and widths/depths, with five shared folds, Adam and 1000 epochs at seed 2026. Activation and hidden-layer initialisation changed together. Width is the number of neurons in each hidden layer.

			Learning rate		
Activation	Hidden layers	Width	0.0003	0.001	0.003
Sigmoid	1	25	0.0498	0.0104	0.0100
Sigmoid	1	50	0.0155	0.0100	0.0100
Sigmoid	1	100	0.0167	0.0100	0.0116
Sigmoid	2	25	0.0098	0.0099	0.0099
Sigmoid	2	50	0.0100	0.0099	0.0099
Sigmoid	2	100	0.0099	0.0099	0.0100
ReLU	1	25	0.0100	0.0103	0.0105
ReLU	1	50	0.0101	0.0104	0.0107
ReLU	1	100	0.0103	0.0105	0.0125
ReLU	2	25	0.0103	0.0104	0.0103
ReLU	2	50	0.0103	0.0103	0.0105
ReLU	2	100	0.0108	0.0107	0.0106
Leaky ReLU	1	25	0.0100	0.0103	0.0106
Leaky ReLU	1	50	0.0101	0.0104	0.0107
Leaky ReLU	1	100	0.0103	0.0105	0.0126
Leaky ReLU	2	25	0.0102	0.0104	0.0102
Leaky ReLU	2	50	0.0103	0.0103	0.0102
Leaky ReLU	2	100	0.0107	0.0105	0.0104

C Use of AI/LLM Tools

C.1 Tools Used

I used OpenAI Codex through the desktop application during Project 2, from 27 October to 6 November 2026, for code implementation, debugging, checks, and report assistance. The exact model version is not recorded. Reused Project 1 code also contains earlier ChatGPT contributions from September 2026; the original tool and contribution credits remain in its source comments.

I completed the project individually. I used LLM assistance during development, including help understanding unfamiliar material, implementing and debugging code, and reviewing my work.

C.2 Contributions to the Report

The levels distinguish language assistance (1), editorial assistance (2), and substantial text generation (3).

Section	Level	Description of use
Abstract	1	Language corrections and improvements to clarity. I selected the main findings and checked the reported numerical values.
Introduction	2	Helped reorganize and improve the structure of my introduction based on my project question.
Theory	1	Provided explanations of theoretical concepts and helped correct language. I checked the mathematical expressions and wrote the theory from my understanding.
Methods	2	Helped review the experimental procedure, identify potential weaknesses, and improve the explanation of the methodology.
Results	1	Assisted with language and presentation. I selected the results and checked numerical values against experimental outputs.
Discussion	2	Helped question interpretations, identify alternative explanations, and improve the structure of the discussion.
Conclusion	1	Minor language corrections and improvements to clarity.
Supplementary material (Appendices A and B)	3	The LLM drafted the appendices using the project report and experimental results as context. I specified the required supplementary content, focusing on experimental settings, verification procedures, and reproducibility. I curated and modified the generated material to include only information relevant to and consistent with the report. I verified the experimental settings, search ranges, numerical results, and reproducibility details against the recorded experiments and outputs.
LLM declaration	3	An LLM drafted the initial declaration, which I reviewed and modified to reflect actual use.

C.3 Contributions to the Code

Project 2 followed the same methodology as Project 1. I specified the scientific structure, expected inputs and outputs, algorithms, formulas, internal logic, and experimental requirements. The LLM generated most of the Python implementation and project-specific notebook code from these specifications. I reviewed, tested, and modified the implementation and requested further functionality and corrections as the project developed. Earlier coursework and Project 1 routines were reused or adapted, with their source and LLM contributions credited separately in the code.

File in `Code/`	Level	Description and verification
`reused_project1.py`	4	Functions reused from Project 1, where the LLM generated most code from my specified inputs, outputs, algorithms, and formulas. Codex extracted the data, scaling, regression, cross-validation, and optimizer routines for Project 2. Their earlier source credits are retained.
`reused_network.py`	4	Codex integrated and adapted earlier coursework forward passes, manual backpropagation, and training loops with Project 1 optimizer updates. It implemented the shared network interfaces, stable cross-entropy, initialization options, activations, weight-only L1/L2 penalties, and regression estimator from my requirements.

File in Code/	Level	Description and verification (continued)
Project2_working.ipynb	4	Codex generated or adapted the Runge regression experiments, shared cross-validation folds, training-only preprocessing, architecture and activation comparisons, regularization studies, checks, and saved-result displays from my specified workflow. Supplied course material and reused methods retain their separate credits.
alpub_v2_feasibility.py	4	Codex implemented AL-PUB archive and image checks, exact-pixel duplicate handling, publication grouping, and the fixed development/test split from my data-handling requirements. The script saves the dataset counts and split records used by the classification workflow.
part_e_classification.py	4	Codex implemented the grouped classification pipeline from my Part e design, reusing the manual network and earlier image preprocessing. This includes development sampling, grouped validation and monitoring, hyperparameter search, early stopping, logistic and scikit-learn comparisons, frozen selection, and final test evaluation.
check_part_c.py	4	Codex generated independent JAX reference objectives and synthetic checks for activation conventions, seeded initialization, and manual regression gradients.
check_part_d.py	4	Codex generated synthetic checks comparing the manual objectives and gradients with independent JAX objectives, including weight-only L1/L2 penalties, bias exclusion, and the L1 convention at zero.
check_part_e.py	4	Codex generated independent 24-class gradient checks and synthetic checks of the classification training loop, update equivalence, early stopping, and best-weight restoration.
present_part_e.py	4	Codex generated the classification result-table presentation from saved experimental outputs. This script does not fit or select models.
calculate_classification_baselines.py	4	Codex generated the analytical image-independent classification baselines from the saved class frequencies and split records, without fitting or simulating models.

The code is classified as Level 4 because the LLM produced most of the final Python text and project-specific notebook additions, including LLM-generated code inherited from Project 1. This file-level classification does not imply that the LLM devised the scientific structure: I designed the workflow, experiments, modelling choices, and much of the mathematical and internal logic. Individual function prompts often resembled Level 2 use, but Level 4 is the most literal description of the submitted files. For the notebook, this classification applies to the assisted project additions, not to unchanged supplied course material. The code comments identify the reused sources and specific contributions.

C.4 Verification and Responsibility

I reviewed the LLM-assisted material and checked the relevant code, calculations, and experimental results. I retained responsibility for the final implementation, scientific analysis, interpretation, and conclusions.

References

Bell-Mathiesen, Eirik (2026). *FYS-STK3155 Project 1: Regression Analysis, Resampling Methods, and Gradient Descent*. Project 1 course report. Report snapshot dated 27 October 2026. Used only as the source of the earlier Project 1 findings identified in the text.

Bergstra, James and Yoshua Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13.10, pp. 281–305. URL: https://jmlr.org/papers/v13/bergstra12a.html.

Bradbury, James et al. (2018). *JAX: composable transformations of Python+NumPy programs*. JAX and jaxlib 0.11.2 were used in the recorded experiments. URL: https://github.com/jax-ml/jax (visited on 10/27/2026).

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Project 2 recommends Chapters 6, 7 and 8. MIT Press. URL: https://www.deeplearningbook.org/ (visited on 10/27/2026).

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. (2020). "Array programming with NumPy". In: *Nature* 585, pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

Hjorth-Jensen, Morten (2026). *Data Analysis and Machine Learning: Lecture Notes*. University of Oslo, FYS-STK3155/FYS4155 course material. Project 2 recommends Chapters 4, 5 and 8. Course source: https://github.com/EducationalMaterialUiO/MachineLearningUiO. URL: https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/chapter8.html (visited on 10/27/2026).

Hunter, John D. (2007). "Matplotlib: A 2D Graphics Environment". In: *Computing in Science & Engineering* 9.3, pp. 90–95. DOI: 10.1109/MCSE.2007.55. URL: https://doi.org/10.1109/MCSE.2007.55.

McKinney, Wes (2010). "Data Structures for Statistical Computing in Python". In: *Proceedings of the 9th Python in Science Conference*, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a. URL: https://doi.org/10.25080/Majora-92bf1922-00a.

Nielsen, Michael A. (2015). *Neural Networks and Deep Learning*. Determination Press. URL: http://neuralnetworksanddeeplearning.com/ (visited on 10/27/2026).

Pedregosa, Fabian, Gael Varoquaux, Alexandre Gramfort, et al. (2011). "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830. URL: https://jmlr.org/papers/v12/pedregosa11a.html.

Raschka, Sebastian, Yuxi (Hayden) Liu, and Vahid Mirjalili (2022a). *Chapter 11: public NumPy multilayer-perceptron and training-loop code*. Consulted for the minibatch training structure, not the grouped cross-entropy stopping protocol. URL: https://github.com/rasbt/machine-learning-book/blob/main/ch11/ch11.py (visited on 11/04/2026).

– (2022b). *Machine Learning with PyTorch and Scikit-Learn*. Project 2 recommends Chapters 11, 12 and 13. Packt Publishing. ISBN: 9781801819312. URL: https://sebastianraschka.com/blog/2022/ml-pytorch-book.html (visited on 10/27/2026).

scikit-learn developers (2026a). *F1 score: metric and averaging definitions*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html (visited on 11/04/2026).

– (2026b). *MLPClassifier: classifier and incremental-training documentation*. Version 1.9.1. Constructor settings and the partial_fit method. URL: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html (visited on 11/05/2026).

– (2026c). *Ridge and Lasso: objective-function documentation*. Ridge objective: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. Used only for library normalisation conventions. URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html (visited on 11/04/2026).

Swindall, Matthew I. (2024). "Advancing Digital Papyrology: Machine Learning and Blockchain Tools for Modernizing the Study of Ancient Greek Manuscripts". PhD thesis. Middle Tennessee State University. URL: https://jewlscholar.mtsu.edu/items/ae9ab25c-1987-4eeb-a859-ee5299b32b5d (visited on 11/04/2026).

Swindall, Matthew I. et al. (2021). "Exploring Learning Approaches for Ancient Greek Character Recognition with Citizen Science Data". In: *2021 IEEE 17th International Conference on eScience*. Original dataset paper. The AL-PUB v2 release and local audit are recorded separately., pp. 128–137. DOI: 10.1109/eScience51609.2021.00023. URL: https://acwio.github.io/pubs/escience2021-AncientLivesModels.pdf.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, et al. (2020). "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17, pp. 261–272. DOI: 10.1038/s41592-019-0686-2. URL: https://doi.org/10.1038/s41592-019-0686-2.

Waskom, Michael L. (2021). "seaborn: statistical data visualization". In: *Journal of Open Source Software* 6.60, p. 3021. DOI: 10.21105/joss.03021. URL: https://doi.org/10.21105/joss.03021.